

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Курсовая работа
По дисциплине
“Информационные системы”

Этап 4

Выполнил:
Стрельбицкий Илья Павлович

Группа: Р3413

Преподаватель:
Мартин Райла

Санкт-Петербург, 2025г

Задание

Этап 4:

- Реализовать уровень представления приложения для осуществления описанных на первом этапе бизнес-процессов.
- Сформировать итоговый отчет, содержащий все предыдущие этапы.
- Провести презентацию проекта.

Этап 1

1. Введение

1.1 Назначение документа

Назначение данного документа — описание функциональных и нефункциональных требований к веб-сервису «Сервис вендинга печати». Документ предназначен для заказчиков, разработчиков, тестировщиков и будет использоваться как основной источник требований на всех этапах разработки.

1.2 Область применения

Разрабатывается веб-сервис для приема заказов на печать и сканирование, их оплаты и выполнения через сеть физических вендинговых аппаратов.

- **Входит в функционал:** управление пользователями и их счетами, оформление и оплата заказов, формирование заданий для аппаратов, мониторинг состояния аппаратов, личный кабинет с историей и файлами, административная панель.
- **Не входит в функционал:** непосредственное управление механизмами вендинговых аппаратов, реализация полноценного платежного шлюза (используется тестовая форма), управление персоналом и финансовый учет предприятия.

1.3 Определения и аббревиатуры

- **Пользователь (Клиент):** конечный пользователь, заказывающий печать или сканирование.
- **Оператор (Сервисный инженер):** пользователь с правами на мониторинг и обслуживание аппаратов.
- **Администратор:** пользователь с правами на управление системой, пользователями и бизнес-правилами.
- **Аппарат (Вендинговый аппарат):** физическое устройство для печати или сканирования.
- **Точка:** физическая локация, где установлены один или несколько аппаратов.
- **Задание (Job):** машинно-читаемая задача, отправляемая аппарату на выполнение.
- **FR:** Функциональное требование.
- **NFR:** Нефункциональное требование.
- **SRS:** Спецификация требований к программному обеспечению.

1.4 Ссылки

1. Задание на курсовую работу — <https://se.ifmo.ru/courses/is/course-project>
2. Документация по Spring Boot Framework — <https://docs.spring.io/spring-boot/index.html>
3. Документация по React JavaScript Library — <https://react.dev/learn>

1.5 Обзор документа

Документ состоит из следующих разделов:

- **Раздел 1:** Введение — назначение, область применения и определения.
- **Раздел 2:** Общее описание — функции продукта, пользователи, зависимости и ограничения.
- **Раздел 3:** Спецификация требований — детальное описание функциональных, нефункциональных требований и требований к интерфейсам.

2. Общее описание

2.1 Функционал продукта

Основной функционал системы включает:

- Регистрацию и аутентификацию пользователей.
- Управление внутренним платежным счетом (кошельком).
- Оформление заказов на печать и сканирование с выбором параметров и точки получения.
- Формирование и хранение корзины неоплаченных заказов.
- Оплату заказов через внутренний счет.
- Генерацию и доставку заданий для вендинговых аппаратов.
- Получение статусов выполнения заданий от аппаратов.
- Предоставление пользователю личного кабинета с историей заказов и файлами.
- Мониторинг состояния аппаратов для операторов и администраторов.

2.2 Описание пользователей

- **Клиент (Пользователь):** обладает базовой компьютерной грамотностью. Цель: быстро и удобно заказать и получить печатные материалы или сканы. Взаимодействует через веб-интерфейс.
- **Оператор/Сервисный инженер:** технический специалист. Цель: отслеживать состояние аппаратов, пополнять расходники, устранять мелкие неисправности. Взаимодействует через веб-интерфейс с расширенными правами.
- **Администратор:** обладает глубокими знаниями о системе. Цель: управление конфигурацией, пользователями, просмотр отчетов и настройка бизнес-логики. Взаимодействует через административную панель.

2.3 Влияющие факторы и зависимости

- Проект является учебным, поэтому для пополнения счета используется тестовая форма, симулирующая работу платежного шлюза.
- Система зависит от выбранных технологий: Spring Boot, React, PostgreSQL.

2.4 Ограничения

- Система должна быть развернута на учебном сервере helios.
- Backend-приложение должно быть упаковано в JAR-файл со встроенным Tomcat-сервером.
- В качестве СУБД должна использоваться PostgreSQL.

3. Спецификация требований

3.1 Функциональные требования

3.1.1 Управление пользователями и аутентификация

- FR-1.1: Система должна предоставлять функционал для регистрации новых пользователей по email и паролю.
- FR-1.2: Система должна предоставлять функционал для аутентификации зарегистрированных пользователей.
- FR-1.3: Система должна разграничивать права доступа на основе ролей.
- FR-1.4: Система должна автоматически назначать роль ROLE_USER новым зарегистрированным пользователям.
- FR-1.5: Система должна предоставлять функционал выхода из системы (logout).

3.1.2 Управление заказами

- FR-2.1: Система должна позволять пользователю создавать заказы двух типов: на печать и на сканирование.
- FR-2.2: Для заказа на печать пользователь должен иметь возможность загружать файлы и выбирать параметры печати (цвет печати, количество копий).
- FR-2.3: Для заказа на сканирование пользователь должен указывать количество страниц.
- FR-2.4: Пользователь должен иметь возможность выбрать точку получения заказа из списка доступных локаций.
- FR-2.5: Система должна рассчитывать и отображать стоимость заказа на основе выбранных параметров до момента оплаты.
- FR-2.6: Неоплаченные заказы должны автоматически помещаться в корзину пользователя.

3.1.3 Управление платежами и кошельком

- FR-3.1: Система должна автоматически создавать внутренний счет (кошелек) для каждого пользователя при регистрации.
- FR-3.2: Система должна предоставлять интерфейс для пополнения счета через тестовую платежную форму.
- FR-3.3: Система должна списывать средства с внутреннего счета пользователя при оплате заказов из корзины.
- FR-3.4: Система должна проверять достаточность средств на счете перед списанием.
- FR-3.5: При неудачной оплате система должна информировать пользователя о причине (недостаточно средств, ошибка платежной системы).

3.1.4 Формирование и управление заданиями для аппаратов

- FR-4.1: Система должна автоматически формировать задание (Job) для соответствующего вендингового аппарата после успешной оплаты заказа.

- FR-4.2: Задание должно содержать все необходимые для выполнения данные (параметры печати/сканирования, файлы для печати, идентификатор заказа).
- FR-4.3: Система должна обеспечивать доставку задания соответствующему аппарату по сети.

3.1.5 Мониторинг и администрирование

- FR-5.1: Система должна предоставлять администраторам и операторам интерфейс для просмотра состояния всех аппаратов.
- FR-5.2: Система должна предоставлять администраторам интерфейс для управления справочной информацией (точки, аппараты, тарифы).
- FR-5.3: Система должна предоставлять администраторам интерфейс для просмотра отчетов по заказам и платежам.

3.1.6 Управление файлами

- FR-6.1: Система должна позволять пользователю загружать файлы для печати в поддерживаемых форматах (JPG, PNG).
- FR-6.2: Система должна предоставлять пользователю возможность просматривать список загруженных для текущего заказа файлов.
- FR-6.3: Система должна позволять пользователю удалять файлы из состава неоплаченного заказа.
- FR-6.4: Система должна автоматически проверять загруженные файлы на соответствие поддерживаемым форматам.

3.1.7 Управление корзиной

- FR-7.1: Система должна предоставлять пользователю интерфейс для просмотра сводки всех неоплаченных заказов в корзине.
- FR-7.2: Система должна позволять пользователю удалять отдельные заказы из корзины.

3.1.8 Мониторинг аппаратов для Администратора

- FR-8.1: Система должна предоставлять Оператору и Администратору интерфейс для просмотра панели со списком всех аппаратов и их ключевых статусов.
- FR-8.2: Система должна предоставлять Оператору и Администратору интерфейс для просмотра детальной информации по выбранному аппарату.
- FR-8.3: Система должна визуально выделять аппараты с критическими статусами (ошибка, отсутствие расходников).

3.1.9 Взаимодействие с вендинговыми аппаратами (Job Management)

- FR-9.1: Система должна присваивать каждому заданию (Job) уникальный идентификатор и статус.
- FR-9.2: Система должна предоставлять API-эндпоинт для приема смены статусов от вендинговых аппаратов.
- FR-9.3: Система должна автоматически обновлять статус заказа в системе пользователя на основе полученных от аппарата статусов задания.
- FR-9.4: Система должна реализовывать механизм повторной отправки задания на аппарат в случае сбоя связи.
- FR-9.5: При невозможности выполнить задание система должна автоматически отменять заказ и возвращать средства на внутренний счет пользователя.
- FR-9.6: Система должна обеспечивать надежное хранение заданий до их полного выполнения или отмены.

3.1.10 Личный кабинет пользователя

- FR-10.1: Система должна предоставлять пользователю возможность просматривать историю всех своих заказов.
- FR-10.2: Система должна предоставлять пользователю возможность просматривать историю операций по внутреннему счету.
- FR-10.3: Система должна предоставлять пользователю возможность скачать отсканированные документы из завершенных заказов на сканирование.

- FR-10.5: Система должна предоставлять пользователю информацию о текущем балансе счета.

3.1.11 Управление статусами аппаратов

- FR-11.1: Система должна получать и обрабатывать статусные сообщения от аппаратов (онлайн, офлайн, выполнение задания, ошибка, уровень расходников).

3.2 Требования к удобству использования

3.2.1 Время обучения

Новый пользователь без предварительной подготовки должен иметь возможность создать и оплатить заказ на печать не более чем за 5 минут.

3.2.2 Время выполнения операций

Переход между основными разделами приложения (главная страница, корзина, личный кабинет) должен осуществляться не более чем за 3 клика.

3.3 Требования к надежности

3.3.1 Доступность

Система должна обеспечивать доступность на уровне 99% в рабочее время (с 08:00 до 22:00).

3.3.2 Целостность данных

Потеря заданий (Jobs) для аппаратов недопустима. Система должна использовать механизмы очередей с повторной доставкой в случае сбоя связи с аппаратом.

3.4 Требования к производительности

3.4.1 Время отклика

Время отклика для 95% API-запросов в условиях нормальной нагрузки (до 1000 одновременных пользователей) не должно превышать 1000 мс.

3.4.2 Обработка файлов

Система должна позволять пользователю одновременно загружать на сервер до 5 файлов общим объемом до 100 МБ.

3.5 Ограничения разработки

3.5.1 Технологический стек

При реализации системы должны быть использованы следующие технологии:

- Backend: Spring Boot (Java).
- Frontend: React (JavaScript).
- База данных: PostgreSQL.
- Сборка: Maven.

3.5.2 Архитектура

Система должна быть реализована по монолитной архитектуре для упрощения разработки и развертывания на сервере helios.

3.6 Интерфейсы

3.6.1 Пользовательские интерфейсы

Веб-интерфейс будет реализован на React. Принципы построения пользовательского интерфейса:

- **Цветовая палитра:** Основные цвета - синий (акцент доверия и технологичности) и белый/серый (нейтральный фон). Зеленый для статусов «успешно», красный для «ошибка».
- **Типографика:** Беззасечный шрифт (например, Roboto или Open Sans) для лучшей читаемости с экрана.
- **Концепция интерфейса:** Приоритет отдается простоте и минимализму. Навигационная панель вверху, основной контент по центру. Формы будут иметь четкую структуру и валидацию.

3.6.2 Аппаратные интерфейсы

Система взаимодействует с вендинговыми аппаратами по сети. Аппараты должны иметь статический IP-адрес или доменное имя для доступа к серверу. Специфические драйверы для управления аппаратами не реализуются, взаимодействие происходит на уровне HTTP и брокера сообщений Kafka.

3.6.3 Программные интерфейсы

- **Внешний платежный шлюз:** Для пополнения счета система интегрируется с тестовой формой, имитирующей API платежного шлюза (например, форма с успешным/неуспешным ответом).
- **СУБД PostgreSQL:** Для хранения всех данных системы используется база данных PostgreSQL, развернутая на сервере helios.

3.6.4 Сетевые интерфейсы

Взаимодействие между клиентом (браузер) и сервером осуществляется по протоколу HTTP. Взаимодействие между сервером и вендинговыми аппаратами осуществляется через RESTful API и Kafka по защищенному каналу (HTTPS/TLS).

3.7 Требования к лицензированию

Разрабатываемый продукт является учебным проектом. Исходный код будет распространяться под лицензией MIT. Данная лицензия является разрешающей, она позволяет свободно использовать, копировать, модифицировать, распространять программное обеспечение при условии указания авторства. Это соответствует учебным целям проекта и не накладывает ограничений на его дальнейшее использование или изучение.

UC-1: Регистрация нового пользователя

ID: UC-1

Краткое описание: Неавторизованный пользователь создает новую учетную запись в системе.

Главные актеры: Пользователь (неавторизованный)

Второстепенные актеры: Нет

Предусловия: Пользователь не аутентифицирован в системе.

Основной поток:

1. Пользователь заполняет форму регистрации (email, пароль).
2. Система проверяет данные и создает учетную запись с ролью ROLE_USER.
3. Система создает внутренний счет пользователя.
4. Система автоматически аутентифицирует пользователя.

Постусловие: Создана учетная запись, пользователь аутентифицирован.

UC-2: Аутентификация пользователя

ID: UC-2

Краткое описание: Зарегистрированный пользователь входит в систему.

Главные актеры: Пользователь

Второстепенные актеры: Нет

Предусловия: Пользователь имеет зарегистрированную учетную запись.

Основной поток:

1. Пользователь вводит учетные данные.
2. Система проверяет данные и создает сеанс.
3. Система перенаправляет в личный кабинет.

Постусловие: Пользователь аутентифицирован.

UC-3: Выход из системы

ID: UC-3

Краткое описание: Аутентифицированный пользователь завершает свой сеанс.

Главные актеры: Пользователь, Администратор, Оператор

Второстепенные актеры: Нет

Предусловия: Пользователь аутентифицирован в системе.

Основной поток:

1. Пользователь инициирует выход.
2. Система завершает сеанс.
3. Система перенаправляет на главную страницу.

Постусловие: Сеанс завершен.

UC-4: Создание заказа на печать

ID: UC-4

Краткое описание: Пользователь создает новый заказ на печать документов.

Главные актеры: Пользователь

Второстепенные актеры: Нет

Предусловия: Пользователь аутентифицирован.

Основной поток:

1. Пользователь загружает файлы и выбирает параметры печати.
2. Пользователь выбирает точку получения.
3. Система рассчитывает стоимость.
4. Пользователь подтверждает заказ.
5. Система помещает заказ в корзину.

Постусловие: Создан неоплаченный заказ в корзине.

UC-5: Создание заказа на сканирование

ID: UC-5

Краткое описание: Пользователь создает новый заказ на сканирование документов.

Главные актеры: Пользователь

Второстепенные актеры: Нет

Предусловия: Пользователь аутентифицирован.

Основной поток:

1. Пользователь указывает параметры сканирования.
2. Пользователь выбирает точку получения.
3. Система рассчитывает стоимость.
4. Пользователь подтверждает заказ.
5. Система помещает заказ в корзину.

Постусловие: Создан неоплаченный заказ в корзине.

UC-6: Пополнение внутреннего счета

ID: UC-6

Краткое описание: Пользователь пополняет баланс своего внутреннего счета.

Главные актеры: Пользователь

Второстепенные актеры: Тестовая платежная система

Предусловия: Пользователь аутентифицирован.

Основной поток:

1. Система перенаправляет на тестовую платежную форму.
2. Пользователь вводит сумму пополнения.
3. Пользователь завершает платеж.
4. Система зачисляет средства на счет.

Постусловие: Баланс счета увеличен.

UC-7: Оплата заказов из корзины

ID: UC-7

Краткое описание: Пользователь оплачивает выбранные заказы, находящиеся в корзине.

Главные актеры: Пользователь

Второстепенные актеры: Система формирования заданий

Предусловия: Пользователь аутентифицирован, в корзине есть неоплаченные заказы.

Основной поток:

1. Пользователь инициирует оплату заказов из корзины.
2. Система проверяет достаточность средств.
3. Система списывает средства и меняет статус заказов на "Оплачено".
4. Система инициирует формирование заданий.

Постусловие: Заказы оплачены, начато выполнение.

UC-8: Формирование задания для аппарата

ID: UC-8

Краткое описание: Система автоматически создает и отправляет задание на вендинговый аппарат после оплаты заказа.

Главные актеры: Система формирования заданий

Второстепенные актеры: Вендинговый аппарат, Пользователь (косвенно)

Предусловия: Заказ успешно оплачен пользователем.

Основной поток:

1. Система выбирает целевой аппарат.
2. Система формирует задание с параметрами.
3. Система отправляет задание на аппарат.
4. Система устанавливает статус "Отправлено на аппарат".

Постусловие: Задание доставлено на аппарат.

UC-9: Просмотр состояния аппаратов

ID: UC-9

Краткое описание: Оператор или Администратор просматривает статусы всех вендинговых аппаратов.

Главные актеры: Оператор, Администратор

Второстепенные актеры: Вендинговые аппараты

Предусловия: Пользователь аутентифицирован и имеет роль Оператора или Администратора.

Основной поток:

1. Пользователь открывает панель мониторинга.
2. Система отображает список аппаратов с статусами.
3. Пользователь просматривает детальную информацию по аппаратам.

Постусловие: Получена информация о состоянии аппаратов.

UC-10: Обновление статуса задания от аппарата

ID: UC-10

Краткое описание: Вендинговый аппарат отправляет статус выполнения задания, система его обрабатывает.

Главные актеры: Вендинговый аппарат

Второстепенные актеры: Пользователь, Система уведомлений

Предусловия: Задание было отправлено на аппарат.

Основной поток:

1. Аппарат отправляет статус задания.
2. Система обновляет статус задания и заказа.
3. При ошибке система инициирует повторную отправку или отмену.

Постусловие: Статус задания и заказа обновлен.

UC-11: Просмотр истории заказов

ID: UC-11

Краткое описание: Пользователь просматривает историю своих заказов.

Главные актеры: Пользователь

Второстепенные актеры: Нет

Предусловия: Пользователь аутентифицирован.

Основной поток:

1. Пользователь открывает историю заказов.
2. Система отображает список заказов.
3. Пользователь просматривает детали заказов.

Постусловие: Получена информация о истории заказов.

UC-12: Скачивание отсканированных документов

ID: UC-12

Краткое описание: Пользователь скачивает результаты выполненного заказа на сканирование.

Главные актеры: Пользователь

Второстепенные актеры: Нет

Предусловия: Пользователь аутентифицирован, заказ на сканирование выполнен.

Основной поток:

1. Пользователь находит выполненный заказ на сканирование.
2. Пользователь инициирует скачивание.
3. Система предоставляет файлы для скачивания.

Постусловие: Пользователь получил отсканированные документы.

UC-16: Управление корзиной

ID: UC-16

Краткое описание: Пользователь просматривает и управляет заказами в корзине.

Главные актеры: Пользователь

Второстепенные актеры: Нет

Предусловия: Пользователь аутентифицирован, в корзине есть заказы.

Основной поток:

1. Пользователь открывает корзину.
2. Пользователь удаляет заказы или оплачивает заказы
3. Система обновляет статусы заказов, либо удаляет их.

Постусловие: Состав корзины обновлен.

Функциональные требования:

№	ID требования	Приоритетность	Трудоемкость, часы	Стабильность
1	FR-1.1	Критическая	8	Высокая
2	FR-1.2	Критическая	6	Высокая
3	FR-1.3	Высокая	12	Средняя
4	FR-1.4	Средняя	4	Высокая
5	FR-1.5	Средняя	3	Высокая
6	FR-2.1	Критическая	10	Высокая
7	FR-2.2	Критическая	16	Средняя
8	FR-2.3	Высокая	8	Высокая
9	FR-2.4	Высокая	6	Средняя
10	FR-2.5	Высокая	12	Средняя
11	FR-2.6	Средняя	5	Высокая
12	FR-3.1	Критическая	6	Высокая
13	FR-3.2	Высокая	10	Средняя
14	FR-3.3	Критическая	8	Высокая
15	FR-3.4	Высокая	4	Высокая
16	FR-3.5	Средняя	6	Средняя
17	FR-4.1	Критическая	14	Высокая
18	FR-4.2	Высокая	8	Средняя
19	FR-4.3	Высокая	10	Средняя
20	FR-5.1	Высокая	12	Средняя
21	FR-5.2	Средняя	16	Низкая
22	FR-5.3	Средняя	14	Низкая
23	FR-6.1	Критическая	10	Высокая
24	FR-6.2	Средняя	4	Высокая
25	FR-6.3	Средняя	3	Высокая
26	FR-6.4	Высокая	6	Высокая
27	FR-7.1	Средняя	5	Высокая
28	FR-7.2	Средняя	3	Высокая
29	FR-8.1	Высокая	8	Средняя
30	FR-8.2	Средняя	6	Средняя
31	FR-8.3	Низкая	4	Средняя
32	FR-9.1	Высокая	5	Высокая
33	FR-9.2	Критическая	12	Высокая
34	FR-9.3	Высокая	8	Средняя

35	FR-9.4	Средняя	10	Низкая
36	FR-9.5	Средняя	6	Средняя
37	FR-9.6	Высокая	7	Высокая
38	FR-10.1	Средняя	6	Высокая
39	FR-10.2	Средняя	5	Высокая
40	FR-10.3	Средняя	4	Высокая
41	FR-10.5	Средняя	2	Высокая
42	FR-11.1	Высокая	10	Средняя

Обоснование оценок:

- **Критическая приоритетность:** Базовые функции, без которых система не может работать
- **Высокая приоритетность:** Важные функции, существенно влияющие на пользовательский опыт
- **Средняя приоритетность:** Полезные функции, но система может работать без них на начальном этапе
- **Низкая приоритетность:** Дополнительные функции "по остаточному принципу"

Трудоемкость: Оценка включает разработку, тестирование и интеграцию

Стабильность: Отражает вероятность изменений требований в процессе разработки

Нефункциональные требования:

№	ID требования	Приоритетность	Трудоемкость, часы	Стабильность
1	3.2.1	Критическая	20	Высокая
2	3.2.2	Высокая	15	Высокая
3	3.3.1	Критическая	25	Средняя
4	3.3.2	Критическая	30	Высокая
5	3.4.1	Высокая	35	Средняя
6	3.4.2	Средняя	12	Высокая
7	3.5.1	Критическая	-	Высокая
8	3.5.2	Высокая	-	Высокая

Пояснения:

3.2.1 (Время обучения) - Критическая приоритетность, так как напрямую влияет на пользовательский опыт. Трудоемкость включает проектирование интуитивного интерфейса и пользовательских сценариев.

3.2.2 (Время выполнения операций) - Высокая приоритетность, требует тщательного проектирования навигации и информационной архитектуры.

3.3.1 (Доступность) - Критическая приоритетность для бизнеса. Высокая трудоемкость обусловлена необходимостью реализации мониторинга, отказоустойчивости и резервного копирования.

3.3.2 (Целостность данных) - Критическая приоритетность, требует реализации механизмов очередей (Kafka), повторных попыток и транзакционности.

3.4.1 (Время отклика) - Высокая приоритетность. Значительная трудоемкость обусловлена необходимостью оптимизации запросов.

3.4.2 (Обработка файлов) - Средняя приоритетность. Требуется реализации валидации и обработки больших файлов.

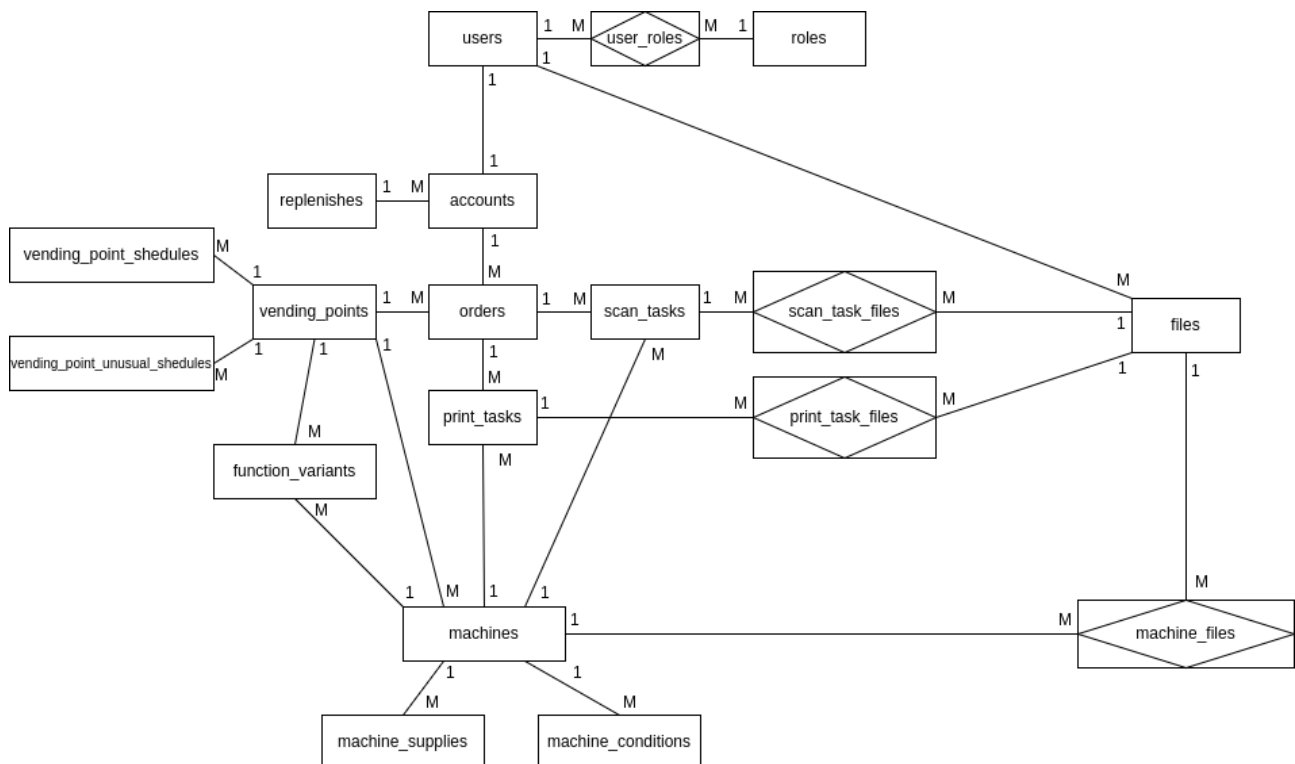
3.5.1 (Технологический стек) - Критическое ограничение (трудоемкость 0 часов, так как это выбор технологий, а не задача на разработку).

3.5.2 (Архитектура) - Высокое ограничение (трудоемкость 0 часов, так как это архитектурное решение).

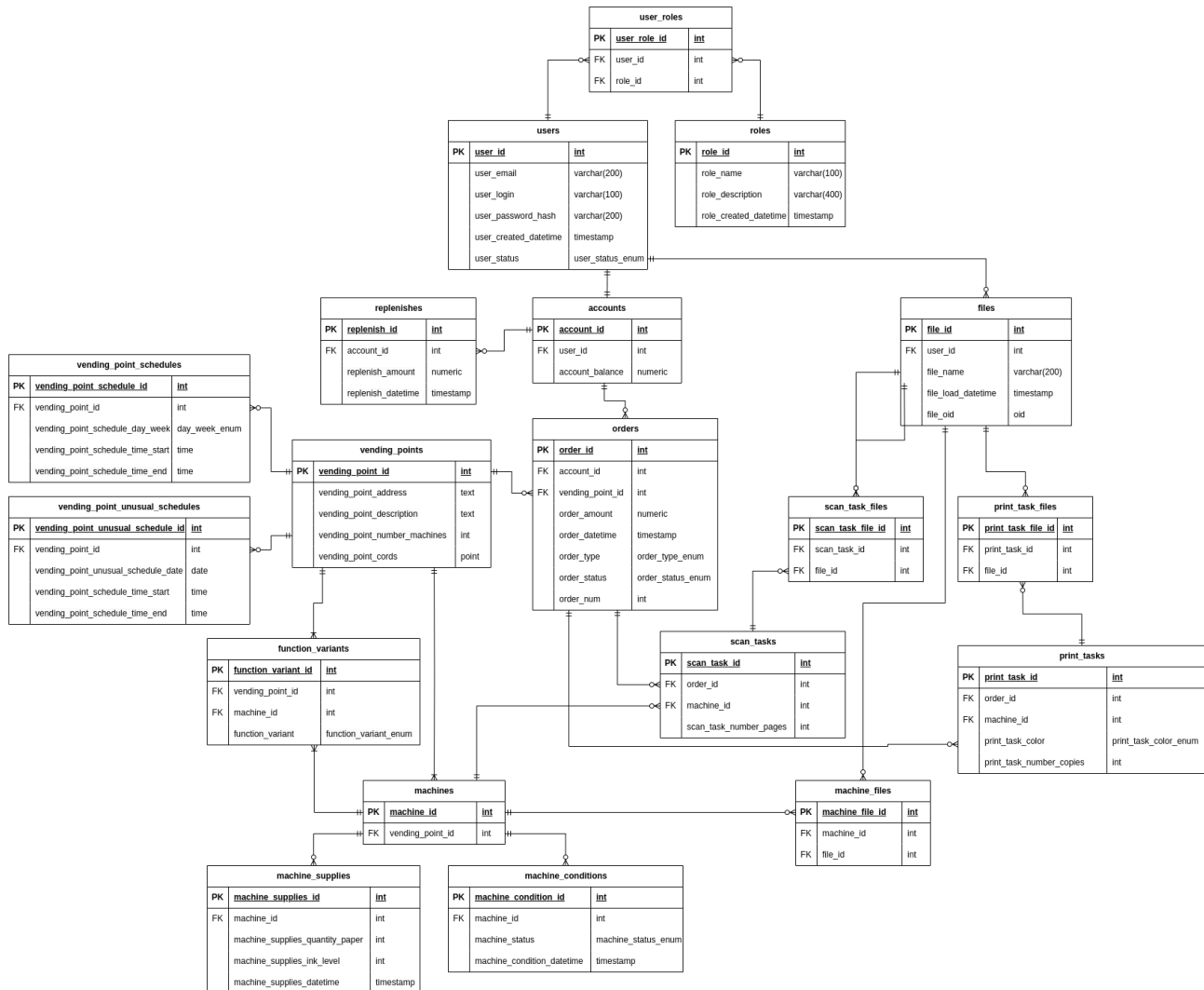
Стабильность: Большинство нефункциональных требований имеют высокую стабильность, так как они определяют основные характеристики системы. Требования к производительности и доступности могут требовать корректировки по результатам тестирования

Этап 2

ER-диаграмма предметной области



Даталогическая модель



Скрипты для создания, удаления базы данных, заполнения базы тестовыми данными

Весь исходный код скриптов, можно найти в репозитории — <https://github.com/stoneshik/is-coursework-2/tree/main>

Скрипт создания базы данных (create.sql):
<https://github.com/stoneshik/is-coursework-2/blob/main/create.sql>

Скрипт очистки базы данных (delete.sql):
<https://github.com/stoneshik/is-coursework-2/blob/main/delete.sql>

Скрипт заполнения базы тестовыми данными в малом объеме (insert.sql):
<https://github.com/stoneshik/is-coursework-2/blob/main/insert.sql>

Скрипт заполнения базы тестовыми данными в большом объеме > 1Гб

(script_generate_data.sql):

https://github.com/stoneshik/is-coursework-2/blob/main/script_generate_data.sql

Pl/pgsql-функции и процедуры

Список функций и процедур:

- `get_role_id_by_name` — вспомогательная функция для создания нового пользователя, получение id роли по ее названию.
- `create_new_user` — функция создания нового пользователя.
- `create_new_user_trigger` — триггер для создания нового пользователя. Сначала пользователю назначается роль user, затем создается новый счет с нулевым балансом и связывается с пользователем.
- `replenish_account` — функция для триггера пополнения счета пользователя.
- `replenish_account_trigger` — триггер для пополнения счета пользователя, значение счета обновляется на сумму предыдущего значения со значением суммы пополнения.

Скрипт создания функций и процедур (triggers.sql):

```
-- триггер для создания нового пользователя
-- 1) Пользователю назначается роль user
-- 2) Создается новый счет с нулевым балансом и связывается с пользователем
CREATE OR REPLACE FUNCTION get_role_id_by_name(varchar(100))
  RETURNS TABLE(role_id integer) AS $$
  SELECT role_id::integer FROM roles WHERE role_name = $1;
$$ LANGUAGE sql;
```

```
CREATE OR REPLACE FUNCTION create_new_user()
  RETURNS trigger AS $$
DECLARE
  user_id_var integer;
  role_id_var integer;
BEGIN
  user_id_var = NEW.user_id;
  role_id_var = get_role_id_by_name('user');
  INSERT INTO user_roles(user_role_id, user_id, role_id) VALUES
    (default, user_id_var, role_id_var);
  INSERT INTO accounts(account_id, user_id, account_balance) VALUES
    (default, user_id_var, 0);
  RETURN NEW;
END
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER create_new_user_trigger
  AFTER INSERT ON users
  FOR EACH ROW
  EXECUTE FUNCTION create_new_user();
```

```
-- триггер для пополнения счета пользователя
-- 1) Значение счета обновляется на сумму предыдущего значения со значением суммы пополнения
CREATE OR REPLACE FUNCTION replenish_account()
  RETURNS trigger AS $$
DECLARE
  account_id_var integer;
  replenish_amount_var numeric;
BEGIN
  account_id_var = NEW.account_id;
  replenish_amount_var = NEW.replenish_amount;
```

```
UPDATE accounts set account_balance = (account_balance + replenish_amount_var) WHERE account_id =  
account_id_var;  
RETURN NEW;  
END  
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER replenish_account_trigger  
AFTER INSERT ON replenishes  
FOR EACH ROW  
EXECUTE FUNCTION replenish_account();
```

Ссылка в репозитории - <https://github.com/stoneshik/is-coursework-2/blob/main/triggers.sql>

Создание индексов, обоснование полезности созданных индексов

Список индексов:

- user_email_index_hash — индекс для быстрого поиска email, для авторизации.
- user_login_index_hash — индекс для быстрого поиска по логину для авторизации.
- order_type_index_btree — индекс для быстрого поиска заказов по типу (печать, сканирование).
- order_status_index_btree — индекс для быстрого поиска заказов по их статусу.
- vending_point_unusual_schedule_date_index_hash — индекс для быстрого поиска расписания по дате
- function_variant_index_btree — индекс для быстрого поиска доступных вариантов по типу варианта (черно-белая печать, цветная печать, сканирование)
- machine_supplies_datetime_index_btree — индекс для ускорения получения последних по времени данных о запасах вендингового аппарата
- machine_condition_datetime_index_btree — индекс для ускорения получения последних по времени данных о состоянии вендингового аппарата

Создание нужных индексов (create_indexes.sql):

-- Создание индексов

```
CREATE INDEX user_email_index_hash ON users USING hash(user_email);  
CREATE INDEX user_login_index_hash ON users USING hash(user_login);
```

```
CREATE INDEX order_type_index_btree ON orders USING btree(order_type);  
CREATE INDEX order_status_index_btree ON orders USING btree(order_status);
```

```
CREATE INDEX vending_point_unusual_schedule_date_index_hash ON vending_point_unusual_schedules  
USING hash(vending_point_unusual_schedule_date);
```

```
CREATE INDEX function_variant_index_btree ON function_variants USING btree(function_variant);
```

```
CREATE INDEX machine_supplies_datetime_index_btree ON machine_supplies USING  
btree(machine_supplies_datetime);  
CREATE INDEX machine_condition_datetime_index_btree ON machine_conditions USING  
btree(machine_condition_datetime);
```

Ссылка в репозитории - <https://github.com/stoneshik/is-coursework-2/blob/main/triggers.sql>

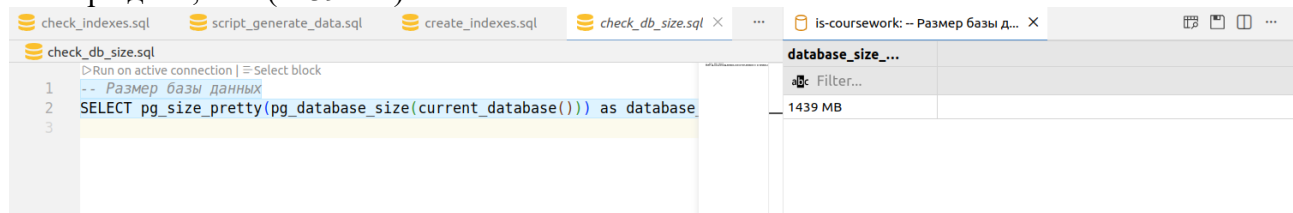
Для доказательства эффективности использования описанных ранее индексов заполним базу данных большим объемом данных (>1 Гб) при помощи скрипта script_generate_data.sql (https://github.com/stoneshik/is-coursework-2/blob/main/script_generate_data.sql).

Запрос проверки размера базы данных:

-- Размер базы данных

```
SELECT pg_size_pretty(pg_database_size(current_database())) as database_size_bytes;
```

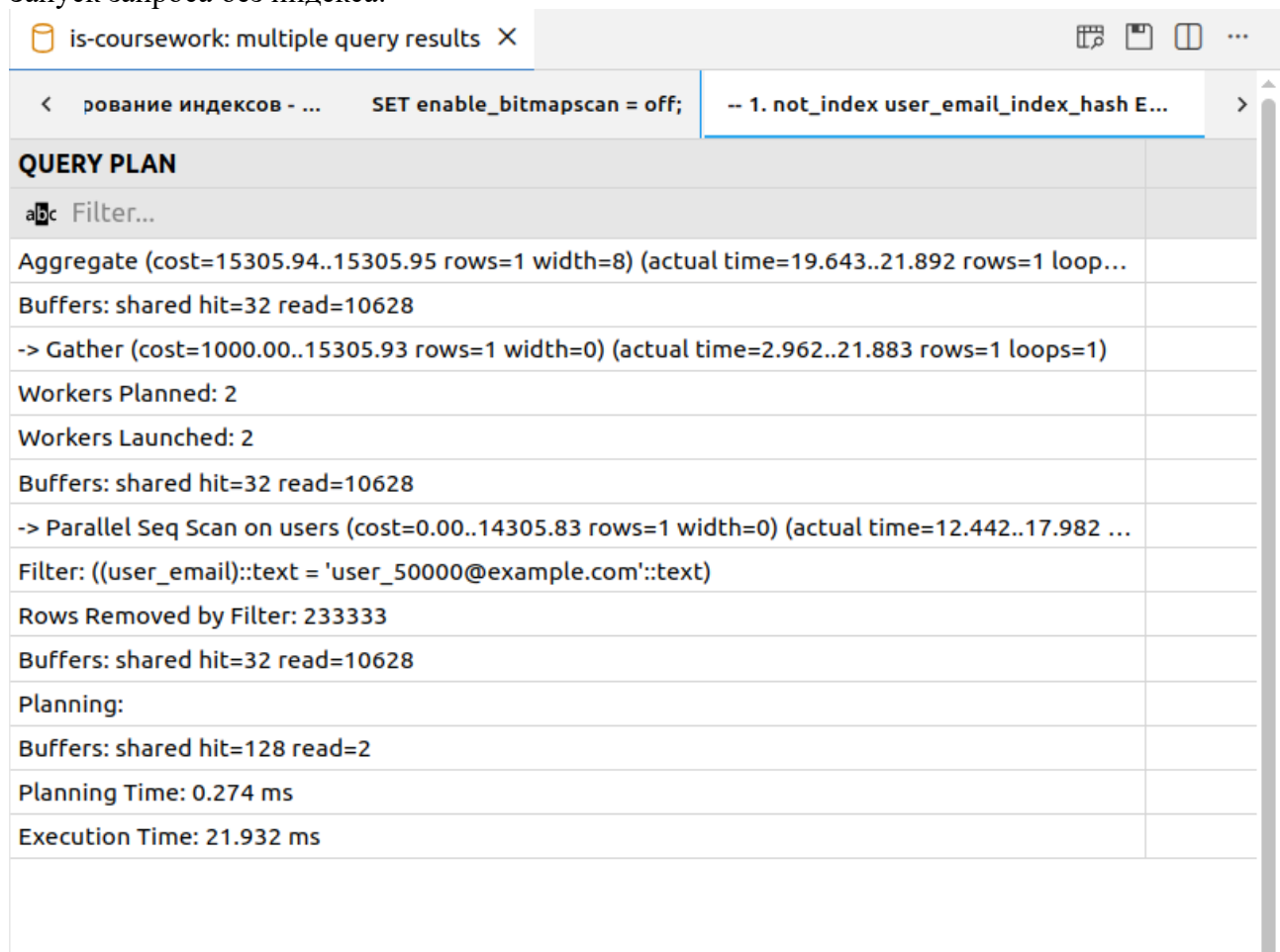
Размер бд – 1,4 Гб (1439 Мб)



Выполним скрипт проверки эффективности индексов, в нем сначала делаются запросы выборки с выключенными индексами и затем точно такие же запросы со включенными индексами. Для анализа выполнения используется Explain Analyze. Также между повторным выполнением запросов очищаем кеш плана выполнения при помощи *DISCARD PLANS*. Скрипт проверки - https://github.com/stoneshik/is-coursework-2/blob/main/check_indexes.sql

1) Проверка user_email_index_hash:

Запуск запроса без индекса:



Запуск запроса с индексом:

is-coursework: multiple query results X	
< второй части... SET enable_bitmapscan = on;	-- 1. index user_email_index_hash EXPLA... -- 2. in >
QUERY PLAN	
Filter...	
Aggregate (cost=4.45..4.46 rows=1 width=8) (actual time=0.053..0.053 rows=1 loops=1)	
Buffers: shared hit=1 read=3	
-> Index Only Scan using users_user_email_key on users (cost=0.42..4.44 rows=1 width=0) (actua...	
Index Cond: (user_email = 'user_50000@example.com'::text)	
Heap Fetches: 0	
- Buffers: shared hit=1 read=3	
Planning Time: 0.052 ms	
Execution Time: 0.069 ms	

2) Проверка user_login_index_hash:

Запуск запроса без индекса:

is-coursework: multiple query results X	
< -- 2. not_index user_login_index_hash EX...	-- 3. not_index order_type_index_btree E... -- 4. not_i >
QUERY PLAN	
Filter...	
Aggregate (cost=15305.94..15305.95 rows=1 width=8) (actual time=20.727..22.491 rows=1 loop...	
Buffers: shared hit=128 read=10532	
-> Gather (cost=1000.00..15305.93 rows=1 width=0) (actual time=3.776..22.484 rows=1 loops=1)	
Workers Planned: 2	
Workers Launched: 2	
- Buffers: shared hit=128 read=10532	
-> Parallel Seq Scan on users (cost=0.00..14305.83 rows=1 width=0) (actual time=13.494..19.124 ...	
Filter: ((user_login)::text = 'user_75000'::text)	
Rows Removed by Filter: 233333	
Buffers: shared hit=128 read=10532	
Planning:	
Buffers: shared hit=3	
Planning Time: 0.091 ms	
Execution Time: 22.509 ms	

Запуск запроса с индексом:

is-coursework: multiple query results X	
< iscan = on; -- 1. index user_email_index_hash EXPLA...	-- 2. index user_login_index_hash EXPLAI...
QUERY PLAN	
Filter...	
Aggregate (cost=4.45..4.46 rows=1 width=8) (actual time=0.021..0.022 rows=1 loops=1)	
Buffers: shared hit=1 read=3	
-> Index Only Scan using users_user_login_key on users (cost=0.42..4.44 rows=1 width=0) (actual...	
Index Cond: (user_login = 'user_75000'::text)	
Heap Fetches: 0	
- Buffers: shared hit=1 read=3	
Planning Time: 0.019 ms	
Execution Time: 0.027 ms	

3) Проверка order_type_index_hash:

Запуск запроса без индекса:

is-coursework: multiple query results X	
< -- 2. not_index user_login_index_hash EX...	-- 3. not_index order_type_index_btree E... -- 4. not_i >
QUERY PLAN	
Filter...	
Finalize Aggregate (cost=76828.45..76828.46 rows=1 width=8) (actual time=174.431..176.429 ro...	
Buffers: shared hit=128 read=47052	
-> Gather (cost=76828.23..76828.44 rows=2 width=8) (actual time=174.360..176.422 rows=3 loo...	
Workers Planned: 2	
Workers Launched: 2	
- Buffers: shared hit=128 read=47052	
-> Partial Aggregate (cost=75828.23..75828.24 rows=1 width=8) (actual time=172.883..172.884 r...	
Buffers: shared hit=128 read=47052	
-> Parallel Seq Scan on orders (cost=0.00..73221.17 rows=1042827 width=0) (actual time=0.013.....	
Filter: (order_type = 'print'::order_type_enum)	
Rows Removed by Filter: 833794	
Buffers: shared hit=128 read=47052	
Planning:	
Buffers: shared hit=56	
Planning Time: 0.130 ms	
Execution Time: 176.454 ms	

Запуск запроса с индексом:

is-coursework: multiple query results X	
< -- 3. index order_type_index_btree EXPL...	-- 4. index order_status_index_btree EXP... -- 5. index >
QUERY PLAN	
Filter...	
Finalize Aggregate (cost=41282.87..41282.88 rows=1 width=8) (actual time=110.327..112.644 ro...	
Buffers: shared hit=6 read=2106	
-> Gather (cost=41282.66..41282.87 rows=2 width=8) (actual time=110.321..112.639 rows=3 loo...	
Workers Planned: 2	
Workers Launched: 2	
- Buffers: shared hit=6 read=2106	
-> Partial Aggregate (cost=40282.66..40282.67 rows=1 width=8) (actual time=108.669..108.670 r...	
Buffers: shared hit=6 read=2106	
-> Parallel Index Only Scan using order_type_index_btree on orders (cost=0.43..37675.59 rows=...	
Index Cond: (order_type = 'print'::order_type_enum)	
Heap Fetches: 0	
Buffers: shared hit=6 read=2106	
Planning Time: 0.029 ms	
Execution Time: 112.670 ms	

4) Проверка order_status_index_hash:

Запуск запроса без индекса:

is-coursework: multiple query results X		🔍 📄 📊 ...
<	ash EX... -- 3. not_index order_type_index_btree E...	-- 4. not_index order_status_index_btree... >
QUERY PLAN		
abc Filter...		
Finalize Aggregate (cost=75939.23..75939.24 rows=1 width=8) (actual time=160.133..161.933 ro...		
Buffers: shared hit=224 read=46956		
-> Gather (cost=75939.02..75939.23 rows=2 width=8) (actual time=160.072..161.927 rows=3 loo...		
Workers Planned: 2		
Workers Launched: 2		
- Buffers: shared hit=224 read=46956		
-> Partial Aggregate (cost=74939.02..74939.03 rows=1 width=8) (actual time=158.236..158.237 r...		
Buffers: shared hit=224 read=46956		
-> Parallel Seq Scan on orders (cost=0.00..73221.17 rows=687140 width=0) (actual time=0.017..1...		
Filter: (order_status = 'completed'::order_status_enum)		
Rows Removed by Filter: 1111468		
Buffers: shared hit=224 read=46956		
Planning:		
Buffers: shared hit=3		
Planning Time: 0.091 ms		
Execution Time: 161.956 ms		

Запуск запроса с индексом:

is-coursework: multiple query results X		🔍 📄 📊 ...
<	-- 3. index order_type_index_btree EXPL...	-- 4. index order_status_index_btree EXP... -- 5. index... >
QUERY PLAN		
abc Filter...		
Finalize Aggregate (cost=27542.41..27542.42 rows=1 width=8) (actual time=74.197..76.236 rows...		
Buffers: shared hit=6 read=1405 written=1070		
-> Gather (cost=27542.19..27542.40 rows=2 width=8) (actual time=74.154..76.230 rows=3 loops=1)		
Workers Planned: 2		
Workers Launched: 2		
Buffers: shared hit=6 read=1405 written=1070		
-> Partial Aggregate (cost=26542.19..26542.20 rows=1 width=8) (actual time=72.367..72.367 ro...		
Buffers: shared hit=6 read=1405 written=1070		
-> Parallel Index Only Scan using order_status_index_btree on orders (cost=0.43..24824.34 rows...		
Index Cond: (order_status = 'completed'::order_status_enum)		
Heap Fetches: 0		
Buffers: shared hit=6 read=1405 written=1070		
Planning Time: 0.066 ms		
Execution Time: 76.254 ms		

5) Проверка vending_point_unusual_schedule_date_index_hash:

Запуск запроса без индекса:

is-coursework: multiple query results X	
< -- 5. not_index vending_point_unusual_s... -- 6. not_index function_variant_index_b... -- 7. not_i >	
QUERY PLAN	
a1c Filter...	
Aggregate (cost=35.39..35.40 rows=1 width=8) (actual time=0.011..0.012 rows=1 loops=1)	
-> Seq Scan on vending_point_unusual_schedules (cost=0.00..35.38 rows=7 width=0) (actual tim...	
Filter: (vending_point_unusual_schedule_date = (CURRENT_DATE - '10 days'::interval))	
Planning:	
Buffers: shared hit=50 read=1	
- Planning Time: 0.140 ms	
Execution Time: 0.021 ms	

Запуск запроса с индексом:

is-coursework: multiple query results X		🔍 📄 📑 ...
< ➤ EXPL...	-- 4. index order_status_index_btree EXP...	-- 5. index vending_point_unusual_sched... >
QUERY PLAN		
abc Filter...		
Aggregate (cost=35.39..35.40 rows=1 width=8) (actual time=0.003..0.003 rows=1 loops=1)		
-> Seq Scan on vending_point_unusual_schedules (cost=0.00..35.38 rows=7 width=0) (actual tim...		
Filter: (vending_point_unusual_schedule_date = (CURRENT_DATE - '10 days'::interval))		
Planning:		
Buffers: shared hit=1		
Planning Time: 0.052 ms		
Execution Time: 0.012 ms		

6) Проверка function_variant_index_hash:

Запуск запроса без индекса:

is-coursework: multiple query results X		🔍 📄 📑 ...
< -- 5. not_index vending_point_unusual_s...	-- 6. not_index function_variant_index_b...	-- 7. not_i >
QUERY PLAN		
abc Filter...		
Aggregate (cost=2811.12..2811.14 rows=1 width=8) (actual time=11.561..11.562 rows=1 loops=1)		
Buffers: shared hit=811		
-> Seq Scan on function_variants (cost=0.00..2686.00 rows=50050 width=0) (actual time=0.009.....		
Filter: (function_variant = 'color_print'::function_variant_enum)		
Rows Removed by Filter: 99937		
- Buffers: shared hit=811		
Planning:		
Buffers: shared hit=33		
Planning Time: 0.076 ms		
Execution Time: 11.570 ms		

Запуск запроса с индексом:

is-coursework: multiple query results X		📄 📁 📂 ...
<	-- 6. index function_variant_index_btree ...	-- 7. index machine_supplies_datetime_i... -- 8. index >
QUERY PLAN		
abc Filter...		
Aggregate (cost=1177.30..1177.31 rows=1 width=8) (actual time=6.655..6.656 rows=1 loops=1)		
Buffers: shared hit=1 read=44 written=44		
-> Index Only Scan using function_variant_index_btree on function_variants (cost=0.29..1052.17 ...)		
Index Cond: (function_variant = 'color_print'::function_variant_enum)		
Heap Fetches: 0		
Buffers: shared hit=1 read=44 written=44		
Planning Time: 0.028 ms		
Execution Time: 6.667 ms		

7) Проверка machine_supplies_datetime_index_btree:

Запуск запроса без индекса:

is-coursework: multiple query results X		📄 📁 📂 ...
<	isual_s... -- 6. not_index function_variant_index_b...	-- 7. not_index machine_supplies_dateti... >
QUERY PLAN		
abc Filter...		
Finalize Aggregate (cost=40469.78..40469.79 rows=1 width=8) (actual time=185.902..187.769 ro...		
Buffers: shared hit=64 read=15860		
-> Gather (cost=40469.57..40469.78 rows=2 width=8) (actual time=185.822..187.764 rows=3 loo...		
Workers Planned: 2		
Workers Launched: 2		
- Buffers: shared hit=64 read=15860		
-> Partial Aggregate (cost=39469.57..39469.58 rows=1 width=8) (actual time=184.312..184.312 r...		
Buffers: shared hit=64 read=15860		
-> Parallel Seq Scan on machine_supplies (cost=0.00..39361.50 rows=43228 width=0) (actual tim...		
Filter: ((machine_supplies_datetime <= now()) AND (machine_supplies_datetime >= (now() - '30 ...		
Rows Removed by Filter: 799142		
Buffers: shared hit=64 read=15860		
Planning:		
Buffers: shared hit=52 read=3		
Planning Time: 0.232 ms		
Execution Time: 187.788 ms		

Запуск запроса с индексом:

is-coursework: multiple query results X		🔍 📄 📑 ...
< -- 6. index function_variant_index_btree ...	-- 7. index machine_supplies_datetime_i...	-- 8. index >
QUERY PLAN		
abc Filter...		
Aggregate (cost=3474.74..3474.75 rows=1 width=8) (actual time=22.719..22.720 rows=1 loops=1)		
Buffers: shared hit=6 read=281 written=281		
-> Index Only Scan using machine_supplies_datetime_index_btree on machine_supplies (cost=0...		
Index Cond: ((machine_supplies_datetime >= (now() - '30 days'::interval)) AND (machine_supplie...		
Heap Fetches: 0		
Buffers: shared hit=6 read=281 written=281		
Planning:		
Buffers: shared hit=4		
Planning Time: 0.081 ms		
Execution Time: 22.733 ms		

8) Проверка machine_condition_datetime_index_btree:

Запуск запроса без индекса:

is-coursework: multiple query results X		🔍 📄 📑 ...
< -- 8. not_index machine_condition_dateti...	DISCARD PLANS;	-- Включаем индексы для е >
QUERY PLAN		
abc Filter...		
Finalize Aggregate (cost=32510.61..32510.62 rows=1 width=8) (actual time=141.992..143.806 ro...		
Buffers: shared hit=64 read=12675		
-> Gather (cost=32510.40..32510.61 rows=2 width=8) (actual time=141.864..143.799 rows=3 loo...		
Workers Planned: 2		
Workers Launched: 2		
- Buffers: shared hit=64 read=12675		
-> Partial Aggregate (cost=31510.40..31510.41 rows=1 width=8) (actual time=140.315..140.316 r...		
Buffers: shared hit=64 read=12675		
-> Parallel Seq Scan on machine_conditions (cost=0.00..31489.00 rows=8559 width=0) (actual ti...		
Filter: ((machine_condition_datetime <= now()) AND (machine_condition_datetime >= (now() - '...		
Rows Removed by Filter: 660283		
Buffers: shared hit=64 read=12675		
Planning:		
Buffers: shared hit=38 read=3		
Planning Time: 0.162 ms		
Execution Time: 143.829 ms		

Запуск запроса с индексом:

is-coursework: multiple query results X	
< _btree ...	-- 7. index machine_supplies_datetime_i...
	-- 8. index machine_condition_datetime_...
QUERY PLAN	
abc Filter...	
Aggregate (cost=690.61..690.62 rows=1 width=8) (actual time=4.051..4.052 rows=1 loops=1)	
Buffers: shared hit=4 read=52 written=52	
-> Index Only Scan using machine_condition_datetime_index_btree on machine_conditions (cost...	
Index Cond: ((machine_condition_datetime >= (now() - '7 days'::interval)) AND (machine_conditi...	
Heap Fetches: 0	
Buffers: shared hit=4 read=52 written=52	
Planning:	
Buffers: shared hit=8	
Planning Time: 0.072 ms	
Execution Time: 4.069 ms	

Таблица сравнения выполнения запросов без индексов и с индексами:

№	С индексами или без индексов	Execution Time (ms)	Разница Execution Time (ms)	Разница Execution Time (%)
1	Без индекса	21,932	21,863	99,69%
	С индексом	0,069		
2	Без индекса	22,509	22,482	99,88%
	С индексом	0,027		
3	Без индекса	176,454	63,784	36,15%
	С индексом	112,670		
4	Без индекса	161,956	85,702	52,92%
	С индексом	76,254		
5	Без индекса	0,021	0,009	42,86%
	С индексом	0,012		
6	Без индекса	11,570	4,903	42,38%
	С индексом	6,667		
7	Без индекса	187,788	165,055	87,89%
	С индексом	22,733		
8	Без индекса	143,829	139,76	97,17%
	С индексом	4,069		

Вывод по использованию индексов:

По планам выполнения запросов видно, то, что индексы используются. При этом все создаваемые индексы значительно оптимизируют время выполнения запросов, поэтому их использование оправдано.

Этап 3

Диаграмма классов системы

Ссылка на диаграмму классов системы — <https://github.com/stoneshik/is-coursework/blob/master/diagram.png>

Исходный код

Исходный код на github — <https://github.com/stoneshik/is-coursework>

Этап 4

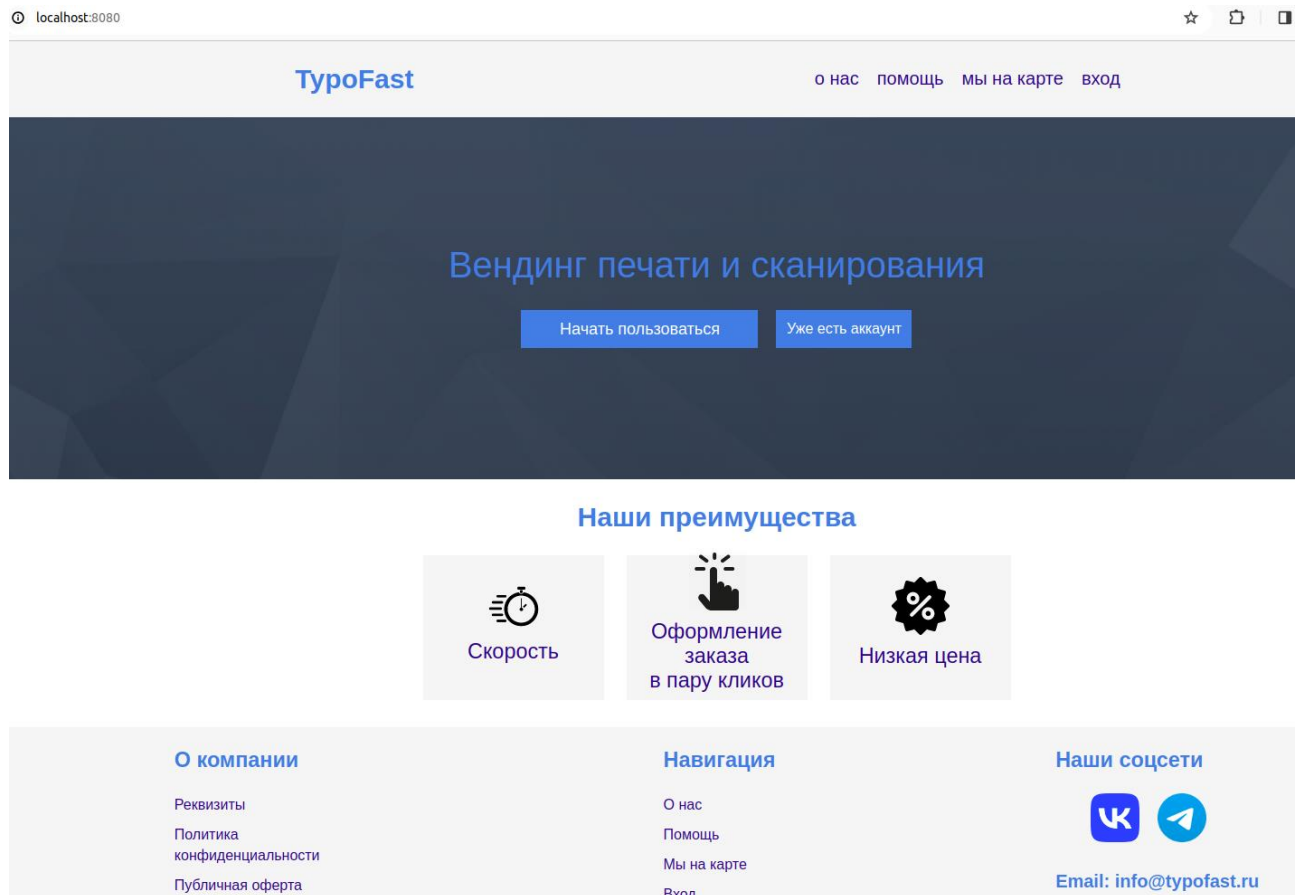
Исходный код

Приложение реализовано с использованием Spring для бэкенда и React для фронтенда.

Исходный код бэкенда на github - <https://github.com/stoneshik/is-coursework>

Исходный код фронтенда на github - <https://github.com/stoneshik/is-coursework-frontend>

Скриншоты работающего приложения



Почта:
aboba@mail.ru

Логин:
aboba

Пароль:

Зарегистрироваться

☐ Я согласен(на) с
[условиями пользования](#)

Войти

О компании

Реквизиты
Политика
конфиденциальности
Публичная оферта

©2023 Все права защищены

Навигация

[О нас](#)
[Помощь](#)
[Мы на карте](#)
[Вход](#)

Наши соцсети



Email: info@typofast.ru

Логин:
aboba

Пароль:

Войти

Зарегистрироваться

О компании

Реквизиты
Политика
конфиденциальности
Публичная оферта

©2023 Все права защищены

Навигация

[О нас](#)
[Помощь](#)
[Мы на карте](#)
[Вход](#)

Наши соцсети



Email: info@typofast.ru

aboba

1096 руб.

Пополнения

Пополнить счет

Выйти

Оплаченные заказы

Номер заказа	Тип заказа	Дата заказа	Сумма заказа	Адрес	
706934	печать	2024-02-16	22 руб.	Невский проспект, 1/4	✖
805612	сканирование	2024-02-16	2 руб.	Невский проспект, 1/4	✖

Новый заказ



Заказ на печать



Заказ на сканирование

Заказ на печать

Выбор места:

Невский проспект, 1/4

Невский пр-кт, 2

Кронвержский проспект, 21/2

Выбрать файлы

Число файлов: 2

Тип печати для всех файлов:

☐ Черно-белая

☐ Цветная

☒ Разное

[third.png](#)

☒ 4/6

☐ Цветная

✖

[fifth.png](#)

☐ 4/6

☒ Цветная

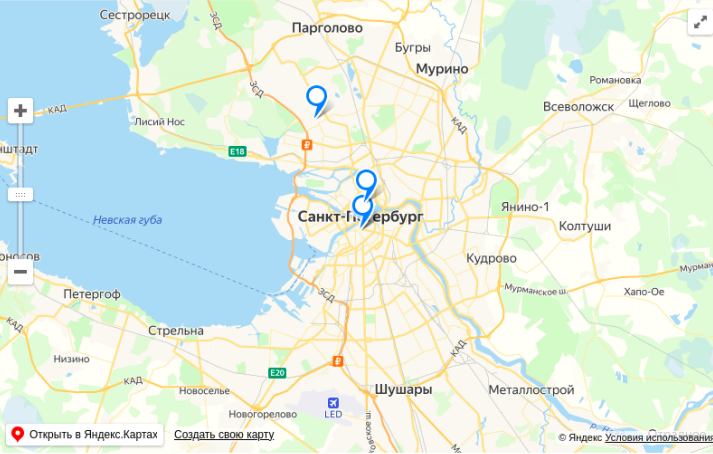
✖

Количество копий печати:

1

Сумма заказа: 22 руб.

Добавить в корзину



Заказ на сканирование

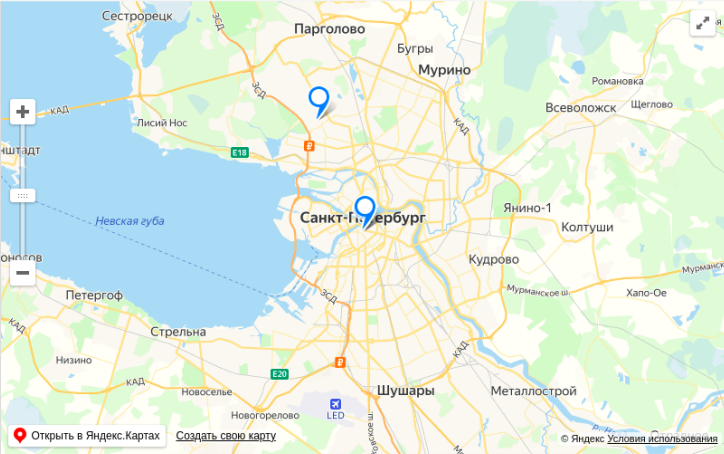
Выбор места:

- Невский проспект, 1/4
- Невский пр-кт, 2
- Комендантский пр-кт, 34

Число страниц:
12

Сумма заказа: 6 руб.

Добавить в корзину



О компании

- Реквизиты
- Политика конфиденциальности
- Публичная оферта

©2023 Все права защищены

Навигация

- О нас
- Помощь
- Мы на карте
- Вход

Наши соцсети



Email: info@typofast.ru

Неоплаченные заказы

	Номер заказа	Тип заказа	Дата заказа	Сумма заказа	Адрес	
☒	706934	печать	2024-02-16	22 руб.	Невский проспект, 1/4	✗
☐	678219	сканирование	2024-02-16	6 руб.	Невский проспект, 1/4	✗
☒	805612	сканирование	2024-02-16	2 руб.	Невский проспект, 1/4	✗

Итого: 24 руб.

Оплатить

О компании

- Реквизиты
- Политика конфиденциальности
- Публичная оферта

©2023 Все права защищены

Навигация

- О нас
- Помощь
- Мы на карте
- Вход

Наши соцсети



Email: info@typofast.ru

Общая информация о заказе

Номер заказа	Тип заказа	Статус	Дата заказа	Сумма заказа	Адрес
805612	сканирование	Оплачен	2024-02-16	2 руб.	Невский проспект, 1/4

Было заказано 4 страниц

О компании

Реквизиты
Политика
конфиденциальности
Публичная оферта

©2023 Все права защищены

Навигация

О нас
Помощь
Мы на карте
Вход

Наши соцсети



Email: info@typofast.ru

Файлы полученные сканированием

Загруженные файлы


[third.png](#)
2024-02-16


[fifth.png](#)
2024-02-16

О компании

Реквизиты
Политика
конфиденциальности
Публичная оферта

©2023 Все права защищены

Навигация

О нас
Помощь
Мы на карте
Вход

Наши соцсети



Email: info@typofast.ru

Пополнения

Номер	Сумма пополнения	Дата пополнения
1	750	2024-02-04
2	200	2024-02-04
3	170	2024-02-04

О компании

Реквизиты
Политика
конфиденциальности
Публичная оферта

©2023 Все права защищены

Навигация

О нас
Помощь
Мы на карте
Вход

Наши соцсети

Email: info@typofast.ru

Сумма пополнения:
120

[Пополнить счет](#)

О компании

Реквизиты
Политика
конфиденциальности
Публичная оферта

©2023 Все права защищены

Навигация

О нас
Помощь
Мы на карте
Вход

Наши соцсети

Email: info@typofast.ru

Видео презентация

Ссылка на видео презентацию —

https://rutube.ru/video/private/d332b572f2605f804da65eab64f62118/?p=m1MMeManw_7R_QgJyzAVrw

Вывод

В ходе выполнения данного этапа курсовой работы был реализован уровень представления для осуществления описанных на первом этапе бизнес-процессов, для реализации уровня представления использовался React JavaScript. Также был сформирован итоговый отчет, содержащий все предыдущие этапы и проведена презентация.

